

FILE2PRINT MONITOR – FULL SYSTEM DESIGN & PROJECT PLAN

Project: File2Print Monitor (Java Desktop App)
Stack: JavaFX + SQLite + WatchService
Connects: Node.js File2Print (uploads folder) ↔ Java Dashboard

TABLE OF CONTENTS

1. Project Overview
2. The Three States
3. System Architecture
4. Database Design
5. Application Flow (Step by Step)
6. Class & Package Design
7. FXML / View Design
8. Full File Tree
9. Responsibility Map (who does what)
10. Screen Flow Diagram
11. Rules & Business Logic
12. Build Order (what to build first)

1. PROJECT OVERVIEW

This Java desktop application monitors the /uploads folder created by the File2Print Node.js web app. When a client submits files through the web interface, File2Print creates a folder inside /uploads named after the client.

This Java app:

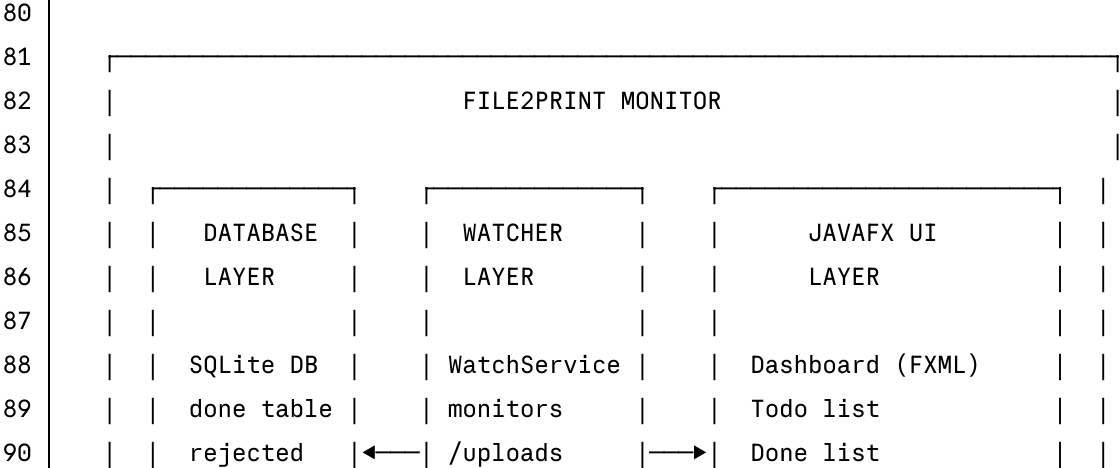
- Watches /uploads in real time using WatchService
- Displays all pending client folders as a TO-DO list
- Lets the owner click a folder to open it in File Explorer for printing
- Lets the owner mark jobs as DONE or REJECTED
- Persists state in a local SQLite database
- On startup, filters out already-processed folders (done/rejected) so only truly pending jobs appear in the live list

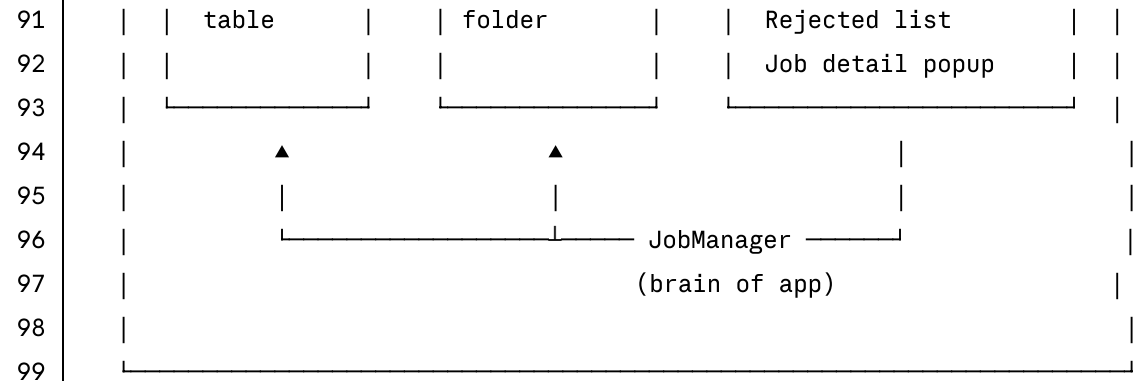
The owner sees a clean dashboard: what needs printing, what is done, what was rejected – all in one place.

```

46 =====
47 2. THE THREE STATES
48 =====
49
50 Every client folder (job) has exactly one state at any time:
51
52 STATE          MEANING          WHERE IT LIVES
53 -----
54 TODO           Folder exists in /uploads,      Not in DB yet – it is live
55                not yet processed. Needs        and detected by WatchService
56                to be printed.                  or found on startup scan.
57
58 DONE           Owner has printed and           Saved in the "done" table
59                marked the job complete.         in SQLite database.
60
61 REJECTED       Owner refused the job           Saved in the "rejected" table
62                (wrong file, bad format,        in SQLite database.
63                client cancelled, etc.)
64
65 FLOW:
66 -----
67 New folder appears in /uploads
68     |
69     ▼
70     TODO (live, shown in dashboard)
71     |
72     |—— Owner clicks DONE   → saved to done table, removed from list
73     |
74     |—— Owner clicks REJECT → saved to rejected table, removed from list
75
76
77 =====
78 3. SYSTEM ARCHITECTURE
79 =====

```





▲
| reads /uploads folder
|

```
graph TD
    subgraph File2Print
        direction TB
        A[File2Print]
        B[Node.js App]
        C[/uploads/]
    end
```

=====

4. DATABASE DESIGN

=====

Database file: file2print.db (SQLite, stored in app root)

Library needed: sqlite-jdbc (add to pom.xml or build.gradle)

TABLE: done

Column	Type	Description
id	INTEGER	Primary key, auto increment
folder_name	TEXT	Name of the client folder
full_path	TEXT	Full path to the folder
client_name	TEXT	Parsed from folder name or data.json
marked_done_at	TEXT	Timestamp when marked done
notes	TEXT	Optional owner notes

TABLE: rejected

Column	Type	Description
id	INTEGER	Primary key, auto increment
folder_name	TEXT	Name of the client folder
full_path	TEXT	Full path to the folder
client_name	TEXT	Parsed from folder name or data.json

```
136 rejected_at TEXT Timestamp when rejected
137 reason TEXT Optional reason for rejection
138
139 QUERIES USED:
140
141 On startup scan:
142 SELECT folder_name FROM done
143 SELECT folder_name FROM rejected
144 → combine both lists → skip these folders → rest goes to TODO list
145
146 When marking done:
147 INSERT INTO done (folder_name, full_path, client_name, marked_done_at)
148
149 When marking rejected:
150 INSERT INTO rejected (folder_name, full_path, client_name, rejected_at)
151
152 For Done tab display:
153 SELECT * FROM done ORDER BY marked_done_at DESC
154
155 For Rejected tab display:
156 SELECT * FROM rejected ORDER BY rejected_at DESC
157
158
159 =====
160 5. APPLICATION FLOW (STEP BY STEP)
161 =====
162
163 — STARTUP FLOW —————
164
165 Step 1: App launches, JavaFX loads Dashboard.fxml
166 Step 2: DashboardController.initialize() is called
167 Step 3: DatabaseManager connects to file2print.db (creates it if not exists)
168 Step 4: DatabaseManager creates done and rejected tables if not exists
169 Step 5: JobManager.loadExistingJobs() runs:
170     a. Scan all folders currently inside /uploads
171     b. Query done table → get list of done folder names
172     c. Query rejected table → get list of rejected folder names
173     d. For each folder in /uploads:
174         - if folder_name is in done list → skip
175         - if folder_name is in rejected list → skip
176         - otherwise → add to TODO ObservableList
177 Step 6: WatcherService.start() begins watching /uploads in background thread
178 Step 7: Dashboard displays the live TODO list to the owner
179
180 — LIVE WATCH FLOW —————
```

```
181
182 Step 1: New client submits files via File2Print web interface
183 Step 2: File2Print creates /uploads/ClientName-DD-MM-YYYY/ folder
184 Step 3: WatchService detects ENTRY_CREATE on /uploads
185 Step 4: WatcherService checks: is this folder_name in done or rejected DB?
186     - Yes → ignore it (already processed, maybe re-uploaded)
187     - No  → Platform.runLater → add to TODO ObservableList
188 Step 5: Dashboard TODO list updates instantly, owner sees new job
189
190 — OWNER ACTION FLOW – MARK AS DONE —————
191
192 Step 1: Owner sees a job in the TODO list
193 Step 2: Owner clicks the job row
194 Step 3: JobDetailPopup.fxml opens showing:
195     - Client folder name
196     - Files inside the folder
197     - OPEN FOLDER button
198     - MARK AS DONE button
199     - REJECT button
200 Step 4: Owner clicks OPEN FOLDER
201     → Desktop.getDesktop().open(folderPath) opens File Explorer
202     → Owner prints the files
203 Step 5: Owner clicks MARK AS DONE
204     → DatabaseManager.insertDone(job)
205     → JobManager removes job from TODO ObservableList
206     → JobManager adds job to DONE ObservableList
207     → Popup closes
208
209 — OWNER ACTION FLOW – REJECT —————
210
211 Step 1: Owner clicks REJECT on a job in the popup
212 Step 2: RejectDialog.fxml opens asking for a reason (optional)
213 Step 3: Owner confirms
214     → DatabaseManager.insertRejected(job, reason)
215     → JobManager removes job from TODO ObservableList
216     → JobManager adds job to REJECTED ObservableList
217     → Popup closes
218
219
220 =====
221 6. CLASS & PACKAGE DESIGN
222 =====
223
224 PACKAGE STRUCTURE:
225 _____
```

```
226 application/
227 |— main/
228 |— controller/
229 |— model/
230 |— service/
231 |— database/
232 |— util/
233
234
235 PACKAGE: application.main
236 PURPOSE: Entry point of the JavaFX application
237
238
239 CLASS: Main.java
240
241 Extends:    javafx.application.Application
242 Purpose:    Launches the JavaFX app, loads Dashboard.fxml
243 Key method: start(Stage stage) – loads FXML, sets up scene, shows window
244 Also:       stop() – called on app close, cancels watcher thread cleanly
245
246
247 PACKAGE: application.controller
248 PURPOSE: JavaFX controllers – one per FXML view
249
250
251 CLASS: DashboardController.java
252
253 Controls:    Dashboard.fxml
254 Purpose:     Main screen controller. Manages the three tab lists.
255 Fields:      @FXML ListView todoList
256              @FXML ListView doneList
257              @FXML ListView rejectedList
258              @FXML Label statusLabel
259              @FXML TabPane tabPane
260 Key methods:
261   initialize()      → startup scan + start watcher + bind lists to UI
262   onJobClicked()    → opens JobDetailPopup when a TODO item is clicked
263   refreshDoneList() → queries DB and reloads done tab
264   refreshRejectedList() → queries DB and reloads rejected tab
265
266 CLASS: JobDetailController.java
267
268 Controls:    JobDetail.fxml
269 Purpose:     Popup showing job details when owner clicks a TODO item
270 Fields:      @FXML Label clientNameLabel
```

```
271         @FXML Label folderPathLabel
272         @FXML ListView filesList
273         @FXML Button openFolderBtn
274         @FXML Button markDoneBtn
275         @FXML Button rejectBtn
276 Key methods:
277     setJob(Job job)      → populates the popup with job data
278     onOpenFolder()      → opens File Explorer at job path
279     onMarkDone()        → calls JobManager.markDone(), closes popup
280     onReject()          → opens RejectDialogController, then marks rejected
281
282 CLASS: RejectDialogController.java
283
284 Controls:    RejectDialog.fxml
285 Purpose:     Small dialog asking owner for rejection reason
286 Fields:      @FXML TextArea reasonInput
287              @FXML Button confirmBtn
288              @FXML Button cancelBtn
289 Key methods:
290     onConfirm()        → returns reason string, closes dialog
291     onCancel()         → closes dialog with no action
292
293
294 PACKAGE: application.model
295 PURPOSE: Plain data classes (POJOs) – no logic, just data
296
297
298 CLASS: Job.java
299
300 Purpose:     Represents one client job (one folder in /uploads)
301 Fields:
302     String folderName    → e.g. "Juan dela Cruz-30-04-2026"
303     String fullPath     → e.g. "C:/uploads/Juan dela Cruz-30-04-2026"
304     String clientName    → parsed from folder name or data.json
305     String dateSubmitted → parsed from folder name or data.json
306     List<String> files    → list of filenames inside the folder
307     String status        → "todo", "done", or "rejected"
308
309 CLASS: DoneJob.java
310
311 Purpose:     Represents a completed job retrieved from the done DB table
312 Fields:
313     String folderName
314     String clientName
315     String markedDoneAt
```

```
316     String notes
317
318 CLASS: RejectedJob.java
319 _____
320 Purpose:     Represents a rejected job from the rejected DB table
321 Fields:
322     String folderName
323     String clientName
324     String rejectedAt
325     String reason
326
327 _____
328 PACKAGE: application.service
329 PURPOSE: Business logic – the brain of the application
330 _____
331
332 CLASS: JobManager.java
333 _____
334 Purpose:     Central manager for all job state. Owns the ObservableLists
335              that the UI binds to. Coordinates between watcher and database.
336 Fields:
337     ObservableList<Job> todoJobs      → bound to todoList in UI
338     ObservableList<DoneJob> doneJobs  → bound to doneList in UI
339     ObservableList<RejectedJob> rejectedJobs → bound to rejectedList in UI
340 Key methods:
341     loadExistingJobs()    → startup scan, filters done/rejected, fills todoJobs
342     addNewJob(Path)       → called by watcher when new folder detected
343                          checks DB first, adds to todoJobs if not processed
344     markDone(Job, notes)  → inserts to DB, moves from todoJobs to doneJobs
345     markRejected(Job, reason) → inserts to DB, moves to rejectedJobs
346     readJobFiles(Path)   → reads files inside a folder, returns List<String>
347     parseClientName(String folderName) → extracts name from folder name string
348
349 CLASS: WatcherService.java
350 _____
351 Purpose:     Runs WatchService on a background daemon thread.
352              Notifies JobManager when new folders appear.
353 Fields:
354     Thread watcherThread
355     WatchService watchService
356     Path uploadsDir
357 Key methods:
358     start()              → registers /uploads, starts daemon thread
359     stop()               → interrupts thread, closes WatchService
360     onFolderCreated(Path) → calls JobManager.addNewJob() via Platform.runLater
```

```
361
362
363 PACKAGE: application.database
364 PURPOSE: All database operations – no UI, no business logic
365
366
367 CLASS: DatabaseManager.java
368
369 Purpose:    Handles all SQLite operations. Single connection to file2print.db
370 Key methods:
371     connect()                → opens connection to file2print.db
372     createTables()           → creates done and rejected tables if missing
373     insertDone(Job, notes)    → INSERT into done table
374     insertRejected(Job, reason) → INSERT into rejected table
375     getAllDoneFolderNames()   → returns List<String> of done folder names
376     getAllRejectedFolderNames() → returns List<String> of rejected names
377     getAllDoneJobs()          → returns List<DoneJob> for Done tab display
378     getAllRejectedJobs()      → returns List<RejectedJob> for Rejected tab
379     close()                   → closes DB connection on app exit
380
381
382 PACKAGE: application.util
383 PURPOSE: Helper utilities used across the app
384
385
386 CLASS: FolderOpener.java
387
388 Purpose:    Opens a folder in the OS file explorer (Windows Explorer)
389 Key method: open(String fullPath) → Desktop.getDesktop().open(new File(path))
390
391 CLASS: DateUtil.java
392
393 Purpose:    Date formatting helpers
394 Key method: now() → returns current timestamp as formatted String
395             parse(String folderName) → extracts date from folder name
396
397 CLASS: AppConfig.java
398
399 Purpose:    Central configuration constants
400 Fields:
401     static String UPLOADS_PATH → path to /uploads folder
402     static String DB_NAME      → "file2print.db"
403
404
405 =====
```

```
406 7. FXML / VIEW DESIGN
407 =====
408
409 FXML FILE          CONTROLLER          PURPOSE
410 -----
411 Dashboard.fxml      DashboardController.java    Main window with 3 tabs
412 JobDetail.fxml      JobDetailController.java      Job popup/detail panel
413 RejectDialog.fxml   RejectDialogController.java    Rejection reason dialog
414
415 -----
416 Dashboard.fxml - LAYOUT DESCRIPTION
417 -----
418
419 Window: full screen, dark background
420
421 Top Bar:
422   - App title label "File2Print Monitor"
423   - Status label (shows "Watching..." or last event)
424   - Total count label (e.g. "3 jobs pending")
425
426 Center: TabPane with 3 tabs
427
428 TAB 1 - TO DO (live)
429   - ListView of todo jobs
430   - Each row shows: client name + date submitted + number of files
431   - Clicking a row opens JobDetail.fxml popup
432   - Green highlight for newest items
433
434 TAB 2 - DONE
435   - ListView of completed jobs
436   - Each row shows: client name + date done
437   - Read only, no actions
438
439 TAB 3 - REJECTED
440   - ListView of rejected jobs
441   - Each row shows: client name + reason + date rejected
442   - Read only, no actions
443
444 Bottom Bar:
445   - Path being watched label
446   - Last updated timestamp
447
448 -----
449 JobDetail.fxml - LAYOUT DESCRIPTION
450 -----
```



```
496 | | | |─ model/
497 | | | |─ Job.java
498 | | | |─ DoneJob.java
499 | | | |─ RejectedJob.java
500 | | | |
501 | | | |─ service/
502 | | | |─ JobManager.java
503 | | | |─ WatcherService.java
504 | | | |
505 | | | |─ database/
506 | | | |─ DatabaseManager.java
507 | | | |
508 | | | |─ util/
509 | | | |─ FolderOpener.java
510 | | | |─ DateUtil.java
511 | | | |─ AppConfig.java
512 | | | |
513 | | | |─ resources/
514 | | | |─ application/
515 | | | |─ Dashboard.fxml
516 | | | |─ JobDetail.fxml
517 | | | |─ RejectDialog.fxml
518 | | | |─ styles.css
519 | | | |
520 |─ file2print.db          ← auto created on first run
521 |─ pom.xml               ← Maven dependencies
522 |─ README.md
```

9. RESPONSIBILITY MAP (WHO DOES WHAT)

QUESTION	ANSWERED BY
Who watches the /uploads folder?	WatcherService.java
Who decides if a job is new?	JobManager.java (checks DB first)
Who stores done/rejected state?	DatabaseManager.java
Who holds the live lists?	JobManager.java (ObservableLists)
Who displays the lists?	DashboardController.java (binds to lists)
Who opens File Explorer?	FolderOpener.java
Who handles button clicks?	JobDetailController.java
Who reads files inside a folder?	JobManager.java (readJobFiles)
Who parses client name from folder?	JobManager.java (parseClientName)
Who configures the uploads path?	AppConfig.java

541 Who launches the app? Main.java

542

543

544 =====

545 10. SCREEN FLOW DIAGRAM

546 =====

547

548 App Launches

549 |

550 ▼

551 Dashboard.fxml

552 [[TODO] [DONE] [REJECT]]

553 |

554 |

555 | • Juan dela Cruz 3 files | ← WatcherService adds here in real time

556 | • Maria Santos 1 file |

557 | • John Smith 5 files |

558 |

559 |

560 |

561 | owner clicks a row

562 ▼

563 JobDetail.fxml (popup)

564 [Juan dela Cruz]

565 | Submitted: 30-04-2026 |

566 |

567 | Files: |

568 | • thesis.pdf |

569 | • photo.jpg |

570 |

571 |

572 | [OPEN FOLDER] |

573 | [MARK AS DONE] [REJECT] |

574 |

575 |

576 | DONE

577 ▼

578 Moves to

579 DONE tab

580 in dashboard

581 | Reason? (optional) |

582 | [CONFIRM][CANCEL] |

583 |

584 ▼

585 Moves to REJECTED tab

```
586
587
588 =====
589 11. RULES & BUSINESS LOGIC
590 =====
591
592 RULE 1 – Startup filter
593     On startup, scan /uploads. Any folder whose name exists in the done
594     or rejected table must NOT appear in the TODO list. Only truly
595     unprocessed folders are shown as TODO.
596
597 RULE 2 – Live filter
598     When WatchService detects a new folder, before adding it to TODO,
599     check the database. If it already exists in done or rejected, ignore it.
600     This handles edge cases where File2Print re-creates an old folder.
601
602 RULE 3 – One state at a time
603     A job can only be in one state. Once marked done or rejected, it is
604     removed from TODO immediately. It cannot be in two states.
605
606 RULE 4 – Database is the source of truth for done/rejected
607     The filesystem is the source of truth for TODO. The database is the
608     source of truth for done and rejected. Never infer state from
609     filesystem alone.
610
611 RULE 5 – Open folder before marking done
612     The OPEN FOLDER button should be clicked first so the owner actually
613     prints the files. Marking as done without opening is allowed but
614     the UI should show OPEN FOLDER as the primary action.
615
616 RULE 6 – Rejection reason is optional
617     The owner can reject without a reason. The reason field in the DB
618     can be empty or null.
619
620 RULE 7 – Daemon thread for watcher
621     WatcherService thread must always be daemon = true so it does not
622     prevent the app from closing.
623
624 RULE 8 – Platform.runLater for all UI updates from watcher
625     Any update to ObservableList or any Label/UI element triggered by
626     WatcherService must go through Platform.runLater().
627
628
629 =====
630 12. BUILD ORDER (WHAT TO BUILD FIRST)
```

Follow this order exactly. Each step depends on the previous.

PHASE 1 – Foundation

1. Create Maven/Gradle project, add dependencies (JavaFX, sqlite-jdbc)
2. Build AppConfig.java → hardcode uploads path and DB name
3. Build Main.java → just loads Dashboard.fxml and shows window
4. Build Dashboard.fxml → empty layout, just the 3 tabs structure

PHASE 2 – Database

5. Build DatabaseManager.java → connect, createTables, basic queries
6. Test: run app, confirm file2print.db is created with correct tables

PHASE 3 – Models

7. Build Job.java
8. Build DoneJob.java
9. Build RejectedJob.java

PHASE 4 – Watcher

10. Build WatcherService.java → watch /uploads, print to console only
11. Test: drop folder in /uploads, confirm console detects it

PHASE 5 – Job Manager

12. Build JobManager.java → ObservableLists + loadExistingJobs()
13. Connect WatcherService → JobManager.addNewJob()
14. Test: confirm TODO list populates on startup and on new folder

PHASE 6 – Dashboard UI

15. Build DashboardController.java → bind lists to ListView in FXML
16. Test: confirm TODO items appear on screen in real time

PHASE 7 – Job Detail

17. Build JobDetail.fxml + JobDetailController.java
18. Wire click on TODO item → opens popup
19. Build FolderOpener.java → wire OPEN FOLDER button
20. Test: click job, popup opens, click open folder, Explorer opens

```
676 PHASE 8 – Done & Reject
677
678     21. Wire MARK AS DONE button → DatabaseManager + JobManager.markDone()
679     22. Build RejectDialog.fxml + RejectDialogController.java
680     23. Wire REJECT button → dialog → DatabaseManager + JobManager.markRejected()
681     24. Test full flow: new folder → TODO → open → done/reject → correct tab
682
683 PHASE 9 – Polish
684
685     25. Build styles.css → dark theme, colors per state
686     26. Add status label updates → "Watching...", "New job arrived!"
687     27. Add job count labels → "3 jobs pending"
688     28. Final test of complete flow
689
690 =====
691 END OF PLAN
692 =====
693
```